

Project Report
On
BOOK RECOMMENDATION SYSTEM

submitted in the partial fulfilment for the award of degree in

Master of Business Administration

Business Analytics

By

Algola Venkateshwar

21MBMB13

Under the guidance of

DR. P. Murugan



School Of Management Studies

University of Hyderabad

DECLARATION

I hereby certify that the work presented in this project report entitle “Book Recommendation System.” is a genuine record of my own work carried out as per requirements for the award of the degree of M.B.A Business Analytics session 2021-23, submitted in School Of Management Studies, University of Hyderabad.

This Work is carried out under the guidance of Dr. P. Murugan, Assistant professor, University of Hyderabad.

I have not submitted the matter personified in the project report for award of any other degree.

Date : _____

Place: _____

M.B.A Business Analytics

Algola Venkateshwar

21MBMB13

CERTIFICATE FROM THE GUIDE

The is to certify that the final project report on “Book Recommendation System.” is a record of bonafide work carried out by Algola Venkateshwar(21mbmb13), as a partial fulfilment of the requirement of MBA Business Analytics at the School of Management Studies, University of Hyderabad.

The report has been reported under my guidance and is record of the bonafide work carried out successfully.

DR. P. Murugan

Assistant Professor

School of Management Studies

University of Hyderabad

Table of Contents

ABSTRACT	5
INTRODUCTION	6
What is recommendation system?	6
Content-based filtering	6
Collaborative filtering.....	7
Hybrid recommendation systems	9
Where it is used?	10
Recommendation system in Book Industry	11
Industries That use recommendation system	11
OBJECTIVES	12
Literature Review	13
CONCEPTUAL FRAMEWORK.....	14
DATA	15
METHODOLOGY	15
ANALYSIS	17
POPULARITY BASED RECOMMENDATION SYSTEM	20
ITEM BASED COLLABORATIVE FILTERING	21
USER BASED COLLABORATIVE FILTERING.....	22
CONCLUSION	25
REFERENCE	26

ABSTRACT

A book recommendation system is an information filtering system that provides personalized recommendations to users based on their reading history, preferences, and other relevant factors. With the rapid growth of digital libraries and online bookstores, book recommendation systems have become increasingly important in helping users discover new books and authors. These systems use a variety of techniques, including collaborative filtering, content-based filtering, and hybrid approaches, to generate recommendations. Collaborative filtering relies on similarities in user behavior to identify books that are likely to be of interest to a user, while content-based filtering focuses on the characteristics of books themselves. Hybrid approaches combine both techniques to improve the accuracy and diversity of recommendations. Despite their potential benefits, book recommendation systems face a number of challenges, including data sparsity, cold-start problems, and the need to balance accuracy and serendipity. In this abstract, we review the state-of-the-art in book recommendation systems, their key components, evaluation metrics, and recent advancements. We also discuss ethical and privacy concerns associated with book recommendation systems and highlight potential solutions to address these issues.

INTRODUCTION

What is recommendation system?

Recommendation Systems There is an extensive class of Web applications that involve predicting user responses to options. Such a facility is called a recommendation system. We shall begin this chapter with a survey of the most important examples of these systems. However, to bring the problem into focus, two good examples of recommendation systems are: 1. Offering news articles to on-line newspaper readers, based on a prediction of reader interests. 2. Offering customers of an on-line retailer suggestions about what they might like to buy, based on their past history of purchases and/or product searches. Recommendation systems use a number of different technologies. We can classify these systems into two broad groups. • Content-based systems examine properties of the items recommended. For instance, if a Netflix user has watched many cowboy movies, then recommend a movie classified in the database as having the “cowboy” genre. • Collaborative filtering systems recommend items based on similarity measures between users and/or items. The items recommended to a user are those preferred by similar users. This sort of recommendation system can use the groundwork laid in Chapter 3 on similarity search and Chapter 7 on clustering. However, these technologies by themselves are not sufficient, and there are some new algorithms that have proven effective for recommendation systems.

Types of Recommendation system

Content-based filtering

Content-based filtering is a type of recommendation system that uses the attributes of items to recommend similar items to users. For example, if a user likes action movies, a content-based filtering system would recommend other action movies with similar attributes such as the same director, cast, or genre.

Here's an example of how a content-based filtering system might work:

User	Item Liked	Attribute 1	Attribute 2	Attribute 3
John	The Matrix	Action	Sci-Fi	Keanu Reeves

User	Item Liked	Attribute 1	Attribute 2	Attribute 3
	Recommendations:			
	John Wick	Action	Keanu Reeves	
	Inception	Sci-Fi		
	Terminator 2	Action	Sci-Fi	

In this example, John liked The Matrix, which has attributes of action, sci-fi, and stars Keanu Reeves. The content-based filtering system then recommends other movies with similar attributes such as John Wick, which is also an action movie that stars Keanu Reeves, Inception, which is a sci-fi movie, and Terminator 2, which is an action/sci-fi movie.

Collaborative filtering

Collaborative filtering is a type of recommendation system that uses the preferences of other users who have similar tastes to recommend items to a particular user.

Here's an example of how collaborative filtering might work:

User	Item Liked	Rating
John	The Matrix	5

User	Item Liked	Rating
Jane	The Matrix	4
Bob	The Matrix	3
	Recommendations:	
	John Wick	
	Inception	
	Interstellar	

In this example, John, Jane, and Bob all liked The Matrix, with John giving it the highest rating of 5, and Bob giving it the lowest rating of 3. The collaborative filtering system then recommends other movies that were highly rated by users who also liked The Matrix, such as John Wick, Inception, and Interstellar.

Hybrid recommendation systems

Hybrid recommendation systems combine multiple recommendation techniques to provide more accurate and diverse recommendations.

Here's an example of how a hybrid recommendation system might work:

User	Item Liked	Attribute 1	Attribute 2	Attribute 3	Rating
John	The Matrix	Action	Sci-Fi	Keanu Reeves	5
Jane	The Matrix	Action	Sci-Fi	Keanu Reeves	4
Bob	The Matrix	Action	Sci-Fi	Keanu Reeves	3
	Recommendations:				
	John Wick	Action	Keanu Reeves		4.5
	Inception	Sci-Fi			4.2
	Interstellar	Sci-Fi			4.0

In this example, John, Jane, and Bob all liked The Matrix, and their ratings were used in collaborative filtering. The content-based filtering system also analyzed the attributes of The Matrix and recommended other movies with similar attributes, such as John.

Where it is used?

Recommendation systems can be used in various industries and applications, some of which are:

1. **E-commerce:** In e-commerce, recommendation systems are used to suggest products to customers based on their browsing and purchase history. For example, Amazon's recommendation system suggests products to customers based on their previous purchases and browsing history. This can increase sales by providing customers with personalized recommendations and encouraging them to purchase additional products.
2. **Streaming services:** Streaming services like Netflix and Spotify use recommendation systems to suggest movies, TV shows, and music to users based on their viewing and listening history. This can help users discover new content and improve their overall experience with the service.
3. **Social media:** Social media platforms like Facebook and Instagram use recommendation systems to suggest content, friends, groups, and pages to users based on their interests and activity. This can increase user engagement and retention by providing relevant content and connections to users.
4. **Travel and hospitality:** In the travel and hospitality industry, recommendation systems can be used to suggest destinations, hotels, and activities to customers based on their preferences and previous bookings. This can improve customer satisfaction and increase bookings by providing personalized recommendations to customers.
5. **Online advertising:** Advertisers use recommendation systems to suggest ads to users based on their browsing and search history. This can increase the effectiveness of advertising campaigns by targeting users with relevant ads.
6. **Healthcare:** In healthcare, recommendation systems can be used to suggest treatments and medications to patients based on their medical history and symptoms. This can improve patient outcomes by providing personalized and effective treatment recommendations.
7. **Education:** Educational institutions can use recommendation systems to suggest courses and learning materials to students based on their interests and performance. This can improve student engagement and learning outcomes by providing personalized recommendations to students.
8. **Financial services:** In financial services, recommendation systems can be used to suggest investment opportunities and financial products to customers based on their risk tolerance and

investment goals. This can increase customer satisfaction and retention by providing personalized recommendations to customers.

Overall, recommendation systems can be used in a variety of industries and applications to improve user experience, increase sales and bookings, improve patient outcomes, and provide personalized recommendations to users.

Recommendation system in Book Industry

Recommendation systems are used in the book industry to help readers discover new books and authors that they may be interested in reading. With so many books available to choose from, it can be overwhelming for readers to find books that match their interests and preferences. By using a recommendation system, readers can receive personalized book recommendations based on their reading history, interests, and behavior.

The use of recommendation systems in the book industry has several benefits for both readers and book sellers. For readers, recommendation systems can help them discover new books and authors that they may not have found otherwise. This can increase reader engagement and satisfaction with the reading experience, and encourage readers to continue to read and explore new genres and authors.

For book sellers, recommendation systems can help increase sales by encouraging readers to purchase books that they may not have found otherwise. By providing personalized recommendations, book sellers can improve customer satisfaction and retention, and build a loyal customer base.

Overall, recommendation systems are used in the book industry to provide personalized book recommendations to readers, increase engagement and sales, and improve the overall reading experience for customers.

Industries That use recommendation system

There are many companies that use recommendation systems to provide personalized recommendations to users. Here are some examples:

- 1 **Amazon:** Amazon is perhaps one of the most well-known companies that uses recommendation systems. The company uses collaborative filtering and other recommendation algorithms to suggest products to customers based on their browsing and purchase history.
- 2 **Netflix:** Netflix uses recommendation systems to suggest movies and TV shows to users based on their viewing history and behavior. The company uses a combination of collaborative filtering, content-based filtering, and other recommendation algorithms.
- 3 **Spotify:** Spotify uses recommendation systems to suggest music to users based on their listening history and behavior. The company uses collaborative filtering and content-based filtering to provide personalized recommendations to users.

- 4 **YouTube:** YouTube uses recommendation systems to suggest videos to users based on their behavior and interests. The company uses a combination of collaborative filtering, content-based filtering, and deep learning algorithms to provide personalized recommendations.
- 5 **LinkedIn:** LinkedIn uses recommendation systems to suggest job postings, groups, and other content to users based on their behavior and interests. The company uses collaborative filtering, content-based filtering, and other recommendation algorithms to provide personalized recommendations to users.

Overall, there are many companies across various industries that use recommendation systems to provide personalized recommendations to users and improve user engagement and satisfaction.

OBJECTIVES

- Develop an analytics-based solution that suggests books to users based on their preferences, behavior, and reading history.
- Process and analyze large volumes of data, including user reviews, ratings, browsing history, and purchase history.
- Apply machine learning algorithms to create personalized profiles for each user and make accurate book recommendations in real-time.
- Enhance the user experience and increase engagement by delivering personalized book recommendations.

Literature Review

In this literature review, we will discuss the algorithms used in recommendation systems and their applications in various fields.

Algorithms Used in Recommendation Systems: Several algorithms are used in recommendation systems, including k-nearest neighbors (k-NN), matrix factorization, and deep learning.

K-NN: The k-nearest neighbors algorithm recommends items based on the similarity of other users' preferences. This algorithm is easy to implement and produces accurate recommendations.

Matrix Factorization: Matrix factorization algorithms are used in collaborative filtering recommendation systems to predict the user's preference for items. This algorithm decomposes the user-item rating matrix into two lower-dimensional matrices, which can then be used to predict the user's preferences.

Deep Learning: Deep learning algorithms are used in content-based recommendation systems to analyze the features of the items and recommend similar items. This algorithm uses artificial neural networks to learn the features of the items and make predictions.

Applications of Recommendation Systems: Recommendation systems have been applied in various fields, including e-commerce, social networking, and music and movie recommendations. In the e-commerce field, recommendation systems have been used to increase sales and improve the customer experience. In the social networking field, recommendation systems have been used to connect users with similar interests. In the music and movie recommendation field, recommendation systems have been used to suggest new songs or movies based on the user's preferences.

to be said recommendation systems have become an integral part of various digital platforms. The algorithms discussed in this literature review provide a framework for the development of a book recommendation system using analytics. This literature review highlights the importance of accurate and personalized recommendations to enhance the user experience and increase engagement, ultimately leading to increased sales for the book retailer.

CONCEPTUAL FRAMEWORK

During my project, I selected a dataset from Kaggle, but I encountered several unexpected issues while working with the data.

Data Sparsity: One of the major challenges in building a book recommendation system is data sparsity. In many cases, users may have very few ratings or reviews, or the number of books that have been rated or reviewed may be small. This makes it difficult to predict user preferences accurately.

Cold Start Problem: The cold start problem occurs when a new user signs up or a new book is added to the system, and there is no historical data available. This makes it challenging to provide recommendations since there is no data to base the recommendations on.

Limited Item Information: The quality and amount of item information available can greatly affect the accuracy of the recommendations. If there is limited information about the books, the system may not be able to provide accurate recommendations.

User Diversity: Users have different preferences, and it is challenging to create a recommendation system that can accurately predict the preferences of a diverse user base. A book recommendation system should be able to handle diverse user preferences and make accurate recommendations.

Scalability: As the number of users and items in the system grows, the scalability of the recommendation system becomes a challenge. The system must be able to handle a large amount of data and provide accurate recommendations in real-time.

System Evaluation: Evaluating the performance of the recommendation system is also a challenge. There are several evaluation metrics available, and it is important to choose the right metric to measure the effectiveness of the recommendation system.

Privacy Concerns: Recommendation systems collect user data to provide personalized recommendations, which can raise privacy concerns. It is important to ensure that the user data is handled securely and privacy is maintained.

Overall, creating a book recommendation system requires overcoming several challenges, including data sparsity, the cold start problem, limited item information, user diversity, scalability, system evaluation, and privacy concerns.

DATA

Source

The dataset has been taken from Kaggle.com

Description

Contains 278,858 users (anonymized but with demographic information) providing 1,149,780 ratings (explicit / implicit) about 271,379 books.

Pictorial view of dataset

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1		user_id	location	age	isbn	rating	book_title	book_auth	year_of_p	publisher	img_s	img_m	img_l	Summary	Language	Category	city	state	country
2	0	2	stockton, ca	18	1.95E+08	0	Classical N	Mark P. O.	2002	Oxford Un	http://ima	http://ima	http://ima	Provides	en	['Social Sci	stockton	california	usa
3	1	8	timmins, o	34.7439	2005018	5	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	timmins	ontario	canada
4	2	11400	ottawa, or	49	2005018	0	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	ottawa	ontario	canada
5	3	11676	n/a, n/a, n	34.7439	2005018	8	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	n/a	n/a	n/a
6	4	41385	sudbury, o	34.7439	2005018	0	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	sudbury	ontario	canada
7	5	67544	toronto, o	30	2005018	8	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	toronto	ontario	canada
8	6	85526	victoria, br	36	2005018	0	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	victoria	british col	canada
9	7	96054	ottawa, or	29	2005018	0	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	ottawa	ontario	canada
10	8	116866	ottawa, ,	34.7439	2005018	9	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	ottawa	,	canada
11	9	123629	kingston, c	34.7439	2005018	9	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	kingston	ontario	canada
12	10	177458	ottawa, or	29	2005018	0	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	ottawa	ontario	canada
13	11	200273	comber, or	34.7439	2005018	8	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	comber	ontario	canada
14	12	210926	guelph, on	34.7439	2005018	9	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	guelph	ontario	canada
15	13	219008	halifax, no	60	2005018	7	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	halifax	nova scoti	canada
16	14	263325	fredericton	27	2005018	6	Clara Calla	Richard Br	2001	HarperFlar	http://ima	http://ima	http://ima	In a small	en	['Actresse	fredericton	new bruns	canada
17	15	8	timmins, o	34.7439	60973129	0	Decision ir	Carlo D'Es	1991	HarperPen	http://ima	http://ima	http://ima	Here, for	en	['1940-194	timmins	ontario	canada
18	16	2954	wichita, ka	71	60973129	8	Decision ir	Carlo D'Es	1991	HarperPen	http://ima	http://ima	http://ima	Here, for	en	['1940-194	wichita	kansas	usa
19	17	152827	ottawa, or	40	60973129	7	Decision ir	Carlo D'Es	1991	HarperPen	http://ima	http://ima	http://ima	Here, for	en	['1940-194	ottawa	ontario	canada
20	18	8	timmins, o	34.7439	3.74E+08	0	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	timmins	ontario	canada
21	19	35704	kansas city	53	3.74E+08	6	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	kansas city	missouri	usa
22	20	83160	oregon cit	65	3.74E+08	0	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	oregon cit	oregon	usa
23	21	110912	milpitas, ci	36	3.74E+08	10	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	milpitas	california	usa
24	22	137688	medford, r	46	3.74E+08	0	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	medford	new york	usa
25	23	157969	denver, co	30	3.74E+08	8	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	denver	colorado	usa
26	24	169187	edmonton	34.7439	3.74E+08	0	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	edmonton	alberta	canada
27	25	192665	vacaville, c	47	3.74E+08	8	Flu: The St	Gina Bari X	1999	Farrar Stra	http://ima	http://ima	http://ima	Describes	en	['Medical']	vacaville	california	usa
		Preprocessed data																	

Fig. Pictorial view of thr dataset

METHODOLOGY

In this chapter, the process followed in the project has been discussed in detail. All the steps that were carried out to achieve the objectives of the project have been discussed below:

All the required libraries that are needed for analysis of the dataset were imported into the Jupyter notebook. As the given dataset involves

- Recommendation systems are becoming increasingly popular as the amount of data generated by businesses and individuals continues to grow. Python is a popular programming language for building recommendation systems, and several libraries are available to simplify the development process. some of the key Python libraries that are commonly used in recommendation systems, including pandas, NumPy, Matplotlib, Scikit-learn, Regular expressions (re), Python Imaging Library (PIL), Requests, and Natural Language Toolkit (NLTK).
- Pandas: Pandas is a popular Python library for data manipulation and analysis. It provides easy-to-use data structures and tools for data cleaning, filtering, and transformation. Pandas is often used in recommendation systems for data pre-processing and feature engineering. For

example, it can be used to extract user behavior data from large datasets and convert it into a format that can be used to train a recommendation model.

- **NumPy:** NumPy is a fundamental Python library for scientific computing that is often used in recommendation systems. It provides powerful tools for array manipulation and mathematical operations, which are important for processing large amounts of data in recommendation systems. For example, NumPy can be used to perform matrix operations on user-item interaction data, which is a common technique used in collaborative filtering recommendation systems.
- **Matplotlib:** Matplotlib is a Python library for creating visualizations, such as graphs and charts. It is often used in recommendation systems to visualize the performance of recommendation models. For example, Matplotlib can be used to create a scatter plot to show the predicted ratings of a recommendation model compared to the actual ratings provided by users.
- **Scikit-learn:** Scikit-learn is a popular Python library for machine learning. It provides a wide range of algorithms for classification, regression, clustering, and dimensionality reduction. Scikit-learn is often used in recommendation systems for building predictive models and evaluating their performance. For example, it can be used to build a collaborative filtering model that predicts user ratings based on their interactions with items.
- **Regular Expressions (re):** Regular expressions are a powerful tool for text processing in Python. They are often used in recommendation systems for text analysis and natural language processing (NLP). For example, regular expressions can be used to extract keywords and topics from user reviews or product descriptions, which can then be used to improve the accuracy of a recommendation model.
- **Python Imaging Library (PIL):** The Python Imaging Library (PIL) is a library for working with images in Python. It is often used in recommendation systems for image analysis and processing. For example, PIL can be used to extract features from product images, such as color and texture, which can then be used to build a content-based recommendation model.
- **Requests:** Requests is a popular Python library for working with HTTP requests. It is often used in recommendation systems for web scraping and data collection. For example, Requests can be used to collect user reviews and ratings from e-commerce websites, which can then be used to train a recommendation model.
- **Natural Language Toolkit (NLTK):** The Natural Language Toolkit (NLTK) is a Python library for NLP. It provides tools for tokenization, stemming, and other NLP tasks. NLTK is often used in recommendation systems for text analysis and processing. For example, it can be used to extract topics and sentiment from user reviews, which can then be used to improve the accuracy of a recommendation model.

ANALYSIS

This chapter contains all the results and their respective interpretations. As discussed in previous chapters, we have taken the dataset from Kaggle

- Installing libraries

```
LOAD & CHECK DATA

import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
import re
from PIL import Image
import requests
import random
from sklearn.feature_extraction.text import CountVectorizer
import nltk
from nltk.corpus import stopwords
```

Fig. Installing libraries

Installing required libraries for manipulating data to get desired result.

- Importing data

The dataset contains various types of information, including details about the books such as their titles, authors, publication dates, genres, and publishers; information about the users, such as their usernames, email addresses, ages, genders, and locations; and reviews or ratings provided by users for the different books, which can reveal their likes, dislikes, opinions, and overall satisfaction with the books.

```
In [3]: books=pd.read_csv("../input/book-recommendation-dataset/Books.csv")
books.head(3)
```

Out[3]:

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	In
0	0195153448	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.com/images/P/019
1	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/000
2	0060973129	Decision in Normandy	Carlo D'Este	1991	HarperPerennial	http://images.amazon.com/images/P/006

Fig. data set snippet

The data contains the book ID, book title, book author, year of publication, and images of the books.

- **Checking for null values**

```
print("Any null values in Books:\n",books.isnull().sum())
print("Any null values in Ratings:\n ",ratings.isnull().sum())
print("Any null values in Users:\n",users.isnull().sum())
```

```
Any null values in Books:
ISBN          0
Book-Title     0
Book-Author    1
Year-Of-Publication 0
Publisher      2
Image-URL-S    0
Image-URL-M    0
Image-URL-L    3
dtype: int64
Any null values in Ratings:
User-ID        0
ISBN           0
Book-Rating    0
dtype: int64
Any null values in Users:
User-ID        0
Location       0
Age           110762
dtype: int64
```

Fig. Null values

Data has some null values that includes Book author has 1 null value, image-URL-L has 3 null values and Age has 110762 values

PREPROCESSING

- **Merging user id and ratings**

```
books_data=books.merge(ratings,on="ISBN")
books_data.head()
```

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	lnr
0	0195153448	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.com/images/P/0195153448
1	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018
2	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018
3	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018
4	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018

Fig: Pre processing

There are two types of files containing information about books and reviews respectively. By merging both of them, we can explore the data clearly and gain an idea about the reviews and user preferences

- **Dropping Null values and other columns**

```
df=books_data.copy()
df.dropna(inplace=True)
df.reset_index(drop=True,inplace=True)
df.drop(columns=["ISBN","Year-Of-Publication","Image-URL-S","Image-URL-M"],axis=1)
df.drop(index=df[df["Book-Rating"]==0].index,inplace=True)
df["Book-Title"]=df["Book-Title"].apply(lambda x: re.sub("[\W_]+"," ",x).strip())
df.head()
```

	Book-Title	Book-Author	Publisher	Image-URL-L	User-ID	Book-Rating
1	Clara Callan	Richard Bruce Wright	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	8	5
3	Clara Callan	Richard Bruce Wright	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	11676	8
5	Clara Callan	Richard Bruce Wright	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	67544	8
8	Clara Callan	Richard Bruce Wright	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	116866	9
9	Clara Callan	Richard Bruce Wright	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	123629	9

Fig: Eliminating of null values and columns

To perform further analysis, we only require the title of the book, book author, publisher, Image-URL-L, user ID, and the book rating given by the user. We do not need any null values or some columns such as book ID, year of publication, Image-URL-S, and Image-URL-M since they are not necessary for analysis.

POPULARITY BASED RECOMMENDATION SYSTEM

- Popularity based recommendation systems are based on the rating of items by all the users.
- Popularity based recommendation systems works with the trend. It basically uses the items which are in trend right now.

```
def popular_books(df,n=100):
    rating_count=df.groupby("Book-Title").count()["Book-Rating"].reset_index()
    rating_count.rename(columns={"Book-Rating": "NumberOfVotes"},inplace=True)

    rating_average=df.groupby("Book-Title")["Book-Rating"].mean().reset_index()
    rating_average.rename(columns={"Book-Rating": "AverageRatings"},inplace=True)

    popularBooks=rating_count.merge(rating_average,on="Book-Title")

    def weighted_rate(x):
        v=x["NumberOfVotes"]
        R=x["AverageRatings"]

        return ((v*R) + (m*C)) / (v+m)

    C=popularBooks["AverageRatings"].mean()
    m=popularBooks["NumberOfVotes"].quantile(0.90)

    popularBooks=popularBooks[popularBooks["NumberOfVotes"] >=250]
    popularBooks["Popularity"]=popularBooks.apply(weighted_rate,axis=1)
    popularBooks=popularBooks.sort_values(by="Popularity",ascending=False)
    return popularBooks[["Book-Title", "NumberOfVotes", "AverageRatings", "Popularity"]]

n=10
top_ten=pd.DataFrame(popular_books(df,10))
fig,ax=plt.subplots(1,10,figsize=(17,5))
fig.suptitle("MOST POPULAR 10 BOOKS",fontsize=40,color="deepskyblue")
for i in range(len(top_ten["Book-Title"].tolist())):
    url=df.loc[df["Book-Title"]==top_ten["Book-Title"].tolist()[i], "Image-URL-L"]
    img=Image.open(requests.get(url,stream=True).raw)
    ax[i].imshow(img)
    ax[i].axis("off")
    ax[i].set_title("RATING: {}".format(round(df[df["Book-Title"]==top_ten["Book-Title"].tolist()[i], "AverageRatings"].values[0],2)))
fig.show()
```

Fig:

I have taken the average rating of the users for books and the number of votes to determine the popularity of the book.

MOST POPULAR BOOKS

MOST POPULAR 10 BOOKS



Fig: Most popular books

These are the most popular books I have found after averaging the ratings and votes of the users.

ITEM BASED COLLABORATIVE FILTERING

```
def item_based(bookTitle):
    bookTitle=str(bookTitle)

    if bookTitle in df["Book-Title"].values:
        rating_count=pd.DataFrame(df["Book-Title"].value_counts())
        rare_books=rating_count[rating_count["Book-Title"]<=200].index
        common_books=df[~df["Book-Title"].isin(rare_books)]

        if bookTitle in rare_books:
            most_common=pd.Series(common_books["Book-Title"].unique()).sample(3)
            print("No Recommendations for this Book 😞 \n ")
            print("YOU MAY TRY: \n ")
            print("{} ".format(most_common[0]), "\n")
            print("{} ".format(most_common[1]), "\n")
            print("{} ".format(most_common[2]), "\n")
        else:
            common_books_pivot=common_books.pivot_table(index=["User-ID"],column=
            title=common_books_pivot[bookTitle]
            recommendation_df=pd.DataFrame(common_books_pivot.corrwith(title).so

            if bookTitle in [title for title in recommendation_df["Book-Title"]]
                recommendation_df=recommendation_df.drop(recommendation_df[recom

            less_rating=[]
            for i in recommendation_df["Book-Title"]:
                if df[df["Book-Title"]==i]["Book-Rating"].mean() < 5:
                    less_rating.append(i)
            if recommendation_df.shape[0] - len(less_rating) > 5:
                recommendation_df=recommendation_df[~recommendation_df["Book-Tit

            recommendation_df=recommendation_df[0:5]
            recommendation_df.columns=["Book-Title","Correlation"]

            fig, ax = plt.subplots(1,5,figsize=(17,5))
            fig.suptitle("WOULD YOU LIKE to TRY THESE BOOKS?",fontsize=40,color=
            for i in range(len(recommendation_df["Book-Title"].tolist())):
                url=df.loc[df["Book-Title"]==recommendation_df["Book-Title"].tol
                img=Image.open(requests.get(url,stream=True).raw)
                ax[i].imshow(img)
                ax[i].axis("off")
                ax[i].set_title("RATING: {} ".format(round(df[df["Book-Title"]==
            fig.show()

        else:
            print("❌ COULD NOT FIND ❌")
```

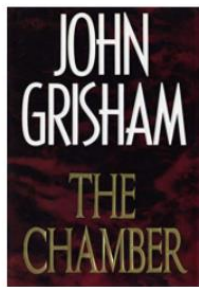
Fig:

Counting the ratings of the books given by users and finding common books, then checking the similarity between the books and past interactions.

Recommending books to user that are similar to the book: Me Talk pretty One Day

```
item_based("Me Talk Pretty One Day")
```

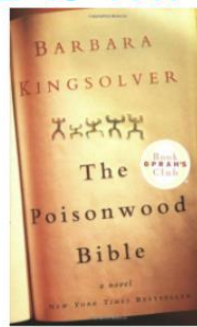
WOULD YOU LIKE to TRY THESE BOOKS?



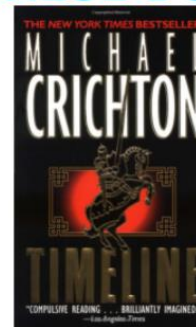
RATING: 7.4



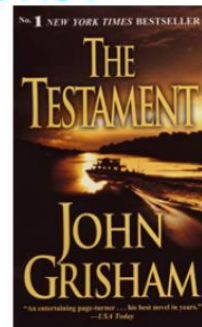
RATING: 7.7



RATING: 8.2



RATING: 7.9



RATING: 7.5

Fig :

The above fig shows that the similar books the user can read.

USER BASED COLLABORATIVE FILTERING

User-based collaborative filtering is a type of recommendation system that suggests items to users based on the preferences and behaviours of similar users.

I have dropped users who have voted less than 200 times, and I am checking similarity using cosine similarity. Then, I am listing similar users and recommending books based on that.

```

new_df=df[df['User-ID'].map(df['User-ID'].value_counts()) > 200] # Drop users who vote Less than 200 times.
users_pivot=new_df.pivot_table(index=["User-ID"],columns=["Book-Title"],values="Book-Rating")
users_pivot.fillna(0,inplace=True)

def users_choice(id):
    users_fav=new_df[new_df["User-ID"]==id].sort_values(["Book-Rating"],ascending=False)[0:5]
    return users_fav

def user_based(new_df,id):
    if id not in new_df["User-ID"].values:
        print("✗ User NOT FOUND ✗")
    else:
        index=np.where(users_pivot.index==id)[0][0]
        similarity=cosine_similarity(users_pivot)
        similar_users=list(enumerate(similarity[index]))
        similar_users = sorted(similar_users,key = lambda x:x[1],reverse=True)[0:5]

        user_rec=[]

        for i in similar_users:
            data=df[df["User-ID"]==users_pivot.index[i[0]]]
            user_rec.extend(list(data.drop_duplicates("User-ID")["User-ID"].values))

    return user_rec

def common(new_df,user,user_id):
    x=new_df[new_df["User-ID"]==user_id]
    recommend_books=[]
    user=list(user)
    for i in user:
        y=new_df[(new_df["User-ID"]==i)]
        books=y.loc[~y["Book-Title"].isin(x["Book-Title"]),:]
        books=books.sort_values(["Book-Rating"],ascending=False)[0:5]
        recommend_books.extend(books["Book-Title"].values)

    return recommend_books[0:5]

user_id=random.choice(new_df["User-ID"].values)
user_choice_df=pd.DataFrame(users_choice(user_id))
user_favorite=users_choice(user_id)
n=len(user_choice_df["Book-Title"].values)
print("■ USER: {} ".format(user_id))

fig,ax=plt.subplots(1,n,figsize=(17,5))
fig.suptitle("YOUR FAVORITE BOOKS",fontsize=40,color="salmon")

for i in range(n):
    url=new_df.loc[new_df["Book-Title"]==user_choice_df["Book-Title"].tolist()[i],"Image-URL-L"][0].values[0]
    img=Image.open(requests.get(url,stream=True).raw)
    ax[i].imshow(img)
    ax[i].axis("off")
    ax[i].set_title("RATING: {} ".format(round(new_df[new_df["Book-Title"]==user_choice_df["Book-Title"].tolist()[i]]["Book-Rating"])))

user_based_rec=user_based(new_df,user_id)
books_for_user=common(new_df,user_based_rec,user_id)
books_for_userDF=pd.DataFrame(books_for_user,columns=["Book-Title"])

fig,ax=plt.subplots(1,5,figsize=(17,5))
fig.suptitle("YOU MAY ALSO LIKE THESE BOOKS",fontsize=40,color="mediumseagreen")
for i in range(5):
    url=new_df.loc[new_df["Book-Title"]==books_for_userDF["Book-Title"].tolist()[i],"Image-URL-L"][0].values[0]
    img=Image.open(requests.get(url,stream=True).raw)
    ax[i].imshow(img)
    ax[i].axis("off")
    ax[i].set_title("RATING: {} ".format(round(new_df[new_df["Book-Title"]==books_for_userDF["Book-Title"].tolist()[i]]["Book-Rating"])))
    fig.show()

```

Fig: Code of user based collaborative filtering

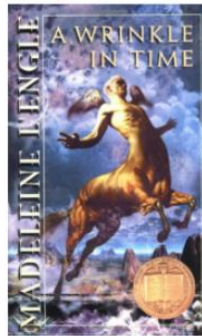
YOUR FAVORITE BOOKS



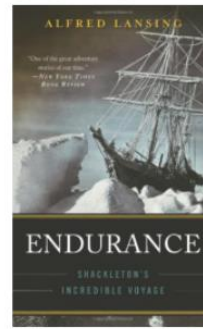
RATING: 8.8



RATING: 10.0



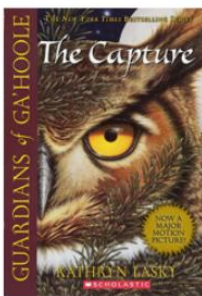
RATING: 9.0



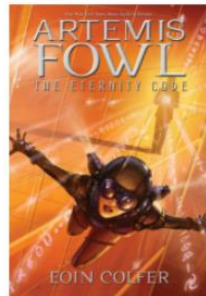
RATING: 8.7

RATING: 10.0

YOU MAY ALSO LIKE THESE BOOKS



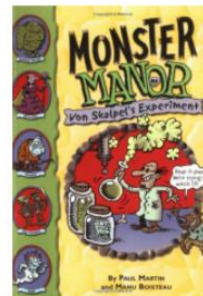
RATING: 10.0



RATING: 8.5



RATING: 9.0



RATING: 10.0



RATING: 8.5

Fig: book suggestion

The above fig shows that recommending books to users who have similar taste

CONCLUSION

Recommendation systems are everywhere these days, with users expecting good product suggestions. While working on a project, I encountered some problems due to the huge and complex data. The major challenge we faced was dealing with a large number of null values. If the data is clean, we can suggest more accurate and relevant products.

Using all the recommendation models, I suggested users go with the recommended products. However, I faced some issues with ratings close to 200, and had to exclude those specific ratings. Understanding user behavior through ratings is difficult since some ratings may seem random and not match the user's taste in books.

In conclusion, a hybrid recommendation model can be used to recommend products that target specific audiences and satisfy both the user and the system's goals.

REFERENCE

<https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00592-5>

<https://www.sciencedirect.com/science/article/pii/S1110866515000341>

<https://ieeexplore.ieee.org/document/8728409>

<https://pandas.pydata.org/>

<https://towardsdatascience.com/deep-dive-into-netflixs-recommender-system-341806ae3b48>